

[CS304] Tutorial 9 - CI/CD tools

What is CI/CD?

CI (Continuous Integration) – automatically build and test every code change pushed to a shared repository.

CD (Continuous Delivery / Deployment) – extend CI so the tested application is always ready to be released (or even deployed automatically).

In this tutorial you will:

- Set up a **Jenkins Pipeline** for a Maven project (Teedy)
- Create a **GitHub Actions** workflow to build and publish artifacts
- (Optional) Trigger Jenkins from GitHub Actions

Part I: Jenkins Pipeline

Install and run Jenkins

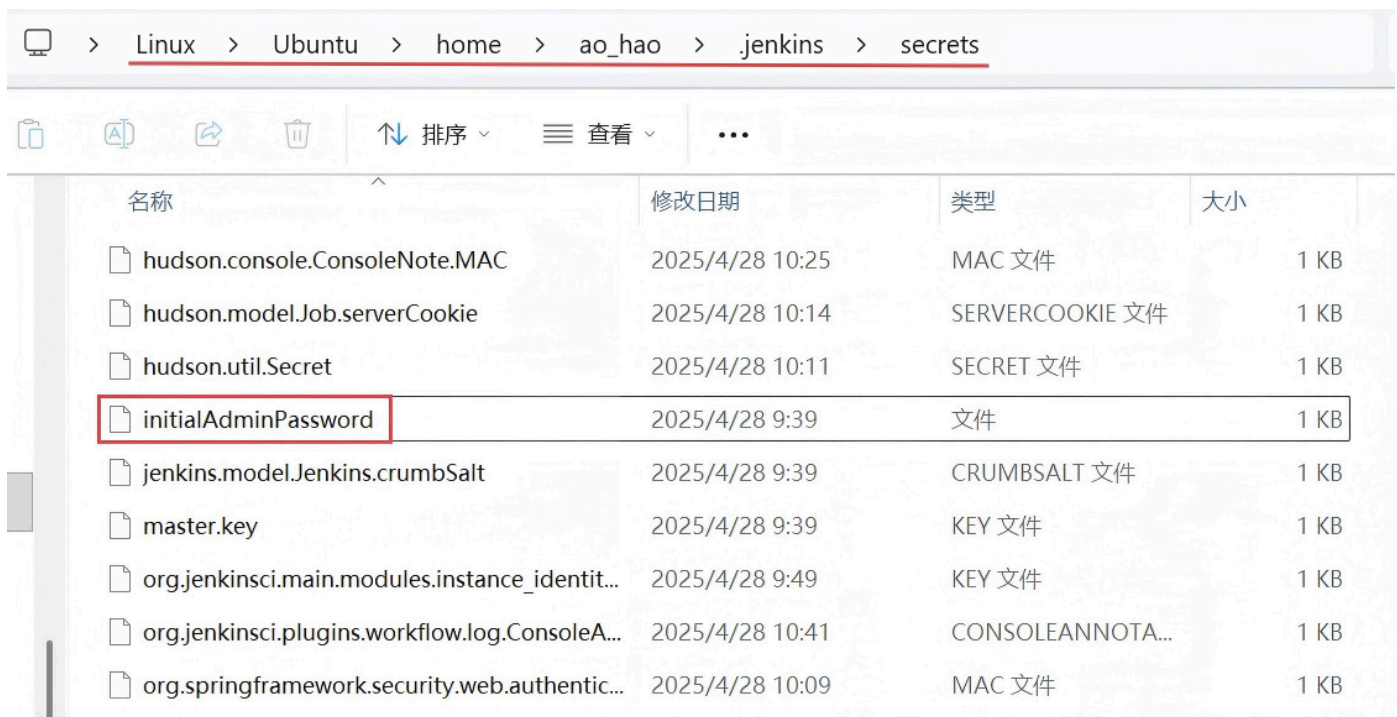
1. Download Jenkins Generic Java package (.war) from jenkins.io/download and save it to wherever you want.
2. Open a terminal, navigate to the directory containing the .war file, and start Jenkins:

```
java -jar jenkins.war --httpPort=8080
```

If your Maven is installed in WSL, run this command inside the WSL terminal.

```
lab@lab: ~/Documents/Softw × + v
Lab@lab:~/Documents/Software/jenkins$ java -jar jenkins.war --httpPort=8080
Running from: /home/lab/Documents/Software/jenkins/jenkins.war
webroot: /home/lab/.jenkins/war
2023-05-12 09:45:15.656+0000 [id=1] INFO winstone.Logger#logInternal:
2023-05-12 09:45:16.638+0000 [id=1] WARNING o.e.j.s.handler.ContextHandle
2023-05-12 09:45:16.702+0000 [id=1] INFO org.eclipse.jetty.server.Serv
7T20:13:20.134Z; git: 1c2636ea05c0ca8de1fffd6ca7f3a98ac084c766d; jvm 17.0.2+8-
2023-05-12 09:45:16.964+0000 [id=1] INFO o.e.j.w.StandardDescriptorPro
did not find org.eclipse.jetty.jsp.JettyJspServlet
2023-05-12 09:45:17.023+0000 [id=1] INFO o.e.j.s.s.DefaultSessionIdMan
2023-05-12 09:45:17.461+0000 [id=1] INFO hudson.WebAppMain#contextInit
.jenkins found at: $user.home/.jenkins
2023-05-12 09:45:17.623+0000 [id=1] INFO o.e.j.s.handler.ContextHandle
87.3,/,file:///home/lab/.jenkins/war/,AVAILABLE}{/home/lab/.jenkins/war}
2023-05-12 09:45:17.637+0000 [id=1] INFO o.e.j.server.AbstractConnecto
```

3. The terminal output will show an **initial admin password**. **Copy this password** – you'll need it to unlock Jenkins.
4. Open you browser and go to <http://localhost:8080>.
5. On the unlock page, the username is `admin` . Paste the initial password you copied.
If you missed the password, you can still find it at
`/home/<your-username>/.jenkins/secrets/initialAdminPassword` .

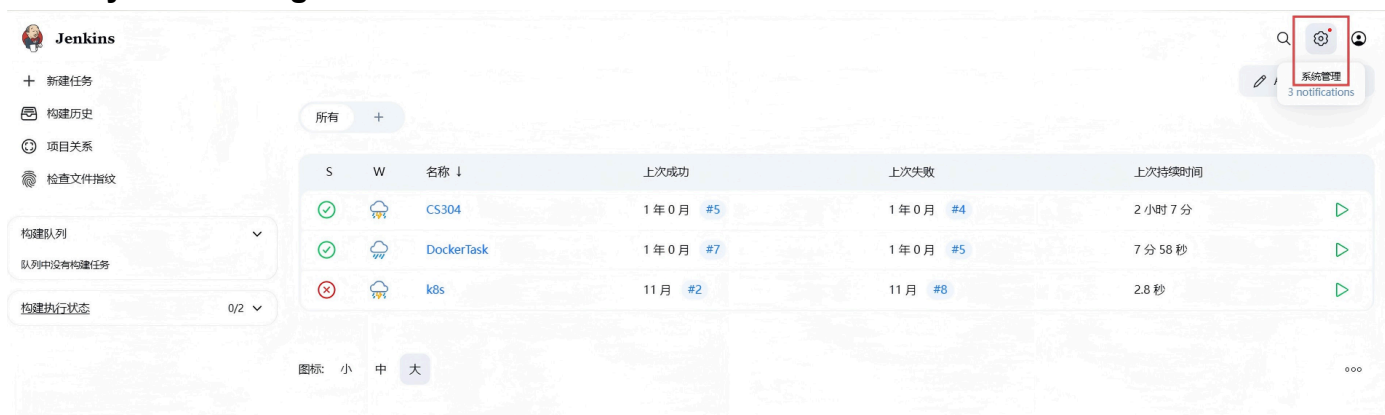


6. Follow the setup wizard. Choose **Install suggested plugins** if you are new to Jenkins.



7. Install the **Maven Integration** plugin:

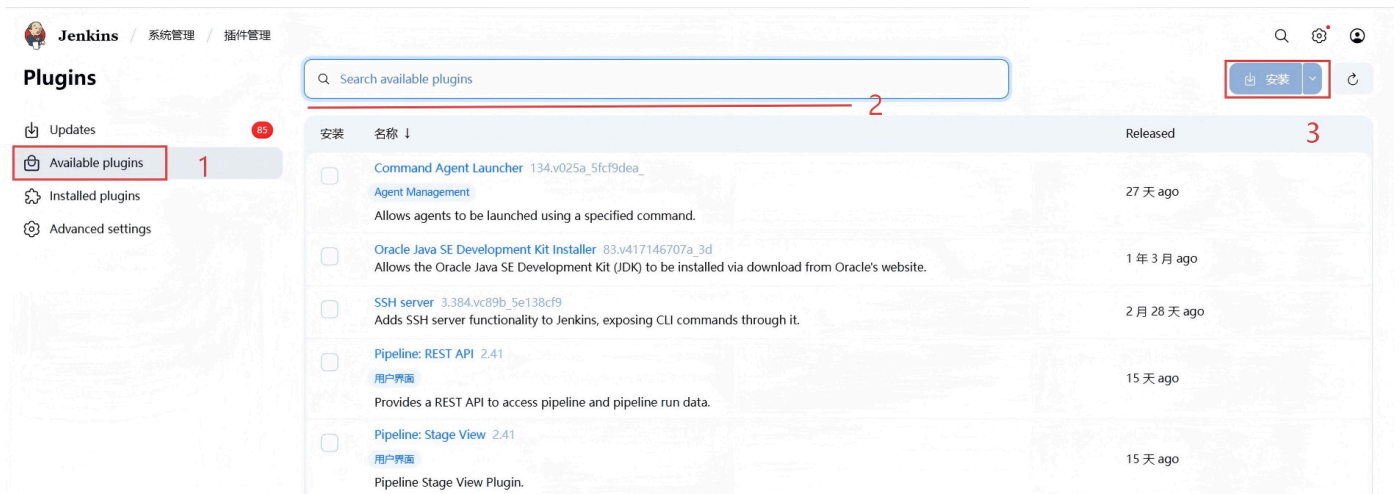
Click **System Management**:



Choose **Plugins Management**:



Go to **Available Plugins** tab, search for “Maven Integration”, select it, and click **Install**.



8. Restart Jenkins after installation.

Create a Jenkins Pipeline project

Now Let's use Jenkins to manage your Teedy project.

1. From the Jenkins dashboard, click **New Item**.



2. Enter a name for your project, select **Pipeline**.



3. In the project configuration, scroll to the **Pipeline** section.

- Under **Definition**, choose **Pipeline script from SCM**.
- Under **SCM**, select **Git**.
- Enter your repository URL (the same Teedy repo you're working on).



If your repository is private, you'll need to provide a GitHub personal access token. Refer to [How to use GitHub Personal Access Token in Jenkins](#).

4. Click **Save** to store the configuration.



Add a Jenkinsfile

Inside your Teedy repository, create a file named `Jenkinsfile` (no extension) at the root. This declarative pipeline defines the entire build process.

```
pipeline {
  agent any

  stages {
    stage('Clean') {
      steps {
        sh 'mvn clean'
      }
    }

    stage('Compile') {
      steps {
        sh 'mvn compile'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test -Dmaven.test.failure.ignore=true'
      }
    }

    stage('PMD') {
      steps {
        sh 'mvn pmd:pmd'
      }
    }

    stage('JaCoCo') {
      steps {
        sh 'mvn jacoco:report'
      }
    }

    stage('Javadoc') {
      steps {
        sh 'mvn javadoc:javadoc'
      }
    }

    stage('Site') {
      steps {
        sh 'mvn site'
      }
    }
  }
}
```

```

    }
}

stage('Package') {
    steps {
        sh 'mvn package -DskipTests'
    }
}

post {
    always {
        archiveArtifacts artifacts: '**/target/site/**/*.*', fingerprint: true
        archiveArtifacts artifacts: '**/target/**/*.*.jar', fingerprint: true
        archiveArtifacts artifacts: '**/target/**/*.*.war', fingerprint: true
        junit '**/target/surefire-reports/*.xml'
    }
}
}

```

Explanation of the pipeline:

- **Stages:** Clean → Compile → Test → PMD → JaCoCo → Javadoc → Site → Package.
- **Stage Javadoc:** this step invokes the **Maven Javadoc Plugin** to automatically generate **API documentation** from Javadoc comments in your Java source code. The output is a set of HTML pages that describe classes, methods, constructors, and fields.
 - **Why include Javadoc generation in CI?**
Ensures documentation is always up-to-date with the latest source code.
Exposes documentation as a build artifact (site report) that can be reviewed or referenced later.
 - **How to make mvn javadoc:javadoc work?**
declare the maven-javadoc-plugin in Teedy's pom.xml (<build> and <reporting>) like jacoco .
- **post { always { ... } }** : Archives test results, site documentation, JAR and WAR files after every build.

Commit and push the Jenkinsfile to your repository:

 COPYING	Initial commit	last year
 Dockerfile	Initial commit	last year
 Jenkinsfile	Create Jenkinsfile	last year
 README.md	Initial commit	last year
 docker-compose.yml	Initial commit	last year
 docs.xml	Initial commit	last year
 pom.xml	Update pom.xml	last year
 rulesets.xml	modify plugins	last year

 README

 Code of conduct

 GPL-2.0 license







License

GPL v2

 Maven CI/CD

failing

Run the Pipeline

1. Go to your pipeline project in Jenkins and click **Build Now**.



- Jenkins will pull the latest commit from your Git repository and execute the stages defined in the Jenkinsfile sequentially.
- Click on the build number (for example, #5) view live progress.



- After the build finishes, you can inspect:



状态

- </> 变更历史
- Console Output
- 编辑构建信息
- 删除构建 '#5'
- Timings
- Git Build Data
- 查看指纹
- Test Result
- Pipeline Overview
- Pipeline Console
- 从指定阶段重新运行
- 回放
- 流水线步骤
- Workspaces
- ← 上一次构建



#5 (2025年4月28日 11:06:13)



Build Artifacts



启动用户admin



This run spent:

- 13 毫秒 waiting;
- 2 小时 7 分 build duration;
- 2 小时 7 分 total from scheduled to completion.



Revision: 9c7823cadb780e5563d4ed84d0c4c2bf1e1d7424

Repository: <https://github.com/zhaoyao0542/Teedy-doc.git>

- refs/remotes/origin/master



Test Result (没有失败)



Changes

1. modify plugins ([details](#) / [githubweb](#))
2. Update pom.xml ([details](#) / [githubweb](#))
3. Update pom.xml ([details](#) / [githubweb](#))

• Console Output – full build log.



状态

- </> 变更历史
- Console Output
- 编辑构建信息
- 删除构建 '#5'
- Timings
- Git Build Data
- 查看指纹
- Test Result
- Pipeline Overview
- Pipeline Console
- 从指定阶段重新运行
- 回放
- 流水线步骤

Console Output

Download

Copy

View as plain text

Skipping 833 KB.. Full Log

```
* @apiPermission admin
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:126: error: unknown tag: apiVersion
* @apiVersion 1.5.0
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:148: error: unknown tag: api
* @api {post} /app/ocr Enable/disable OCR
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:149: error: unknown tag: apiName
* @apiName PostAppOcr
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:150: error: unknown tag: apiGroup
* @apiGroup App
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:151: error: unknown tag: apiParam
* @apiParam {Boolean} enabled If true, enable OCR
^
/home/ao_hao/.jenkins/workspace/CS304/docs-web/src/main/java/com/sismics/docs/rest/resource/AppResource.java:152: error: unknown tag: apiError
* @apiError {client} ForbiddenError Access denied
^
```

- **Workspace** – files checked out by Jenkins.
- **Pipeline Overview** - A visual representation of all stages defined in the Jenkinsfile. Each stage is color-coded (green for success, red for failure) and shows its execution duration. It gives an at-a-glance summary of the build flow, and clicking any stage takes you directly to its detailed console log.



Jenkins / CS304 / #5 / Pipeline Console

成功 1 年 0 月 ago in 2 小时 7 分

Rebuild Overview Configure

Search

Checkout SCM Clean **Compile** Test PMD JaCoCo Javadoc Site Package Post Actions

mvn compile 8 秒

```
0 + mvn compile
1 [INFO] Scanning for projects...
2 [INFO] -----
3 [INFO] Reactor Build Order:
4 [INFO]
5 [INFO] Docs Parent [pom]
6 [INFO] Docs Core [jar]
7 [INFO] Docs Web Commons [jar]
8 [INFO] Docs Web [war]
9 [INFO]
10 [INFO] -----< com.sismics.docs:docs-parent >-----
11 [INFO] Building Docs Parent 1.12-SNAPSHOT [1/4]
12 [INFO] -----[ pom ]-----
```

- **Build Artifacts** – archived JARs, WARs, and site documentation (HTML) available for download.

/ target / site /

apidocs css images jacoco-aggregate

cpd.html	2025年4月25日 下午3:49:48	13.46 KB		
dependency-convergence.html	2025年4月25日 下午3:49:52	3.56 KB		
dependency-info.html	2025年4月25日 下午3:49:52	3.64 KB		
dependency-management.html	2025年4月25日 下午3:49:52	28.11 KB		
distribution-management.html	2025年4月25日 下午3:49:52	3.79 KB		
index.html	2025年4月25日 下午3:49:52	4.21 KB		
modules.html	2025年4月25日 下午3:49:52	4.04 KB		
plugin-management.html	2025年4月25日 下午3:49:52	4.47 KB		
plugins.html	2025年4月25日 下午3:49:52	7.02 KB		
pmd.html	2025年4月25日 下午3:49:48	63.44 KB		
project-info.html	2025年4月25日 下午3:49:52	5.86 KB		
project-reports.html	2025年4月25日 下午3:49:52	4.00 KB		
scm.html	2025年4月25日 下午3:49:52	3.95 KB		
summary.html	2025年4月25日 下午3:49:52	4.53 KB		
surefire-report.html	2025年4月25日 下午3:49:48	66.88 KB		

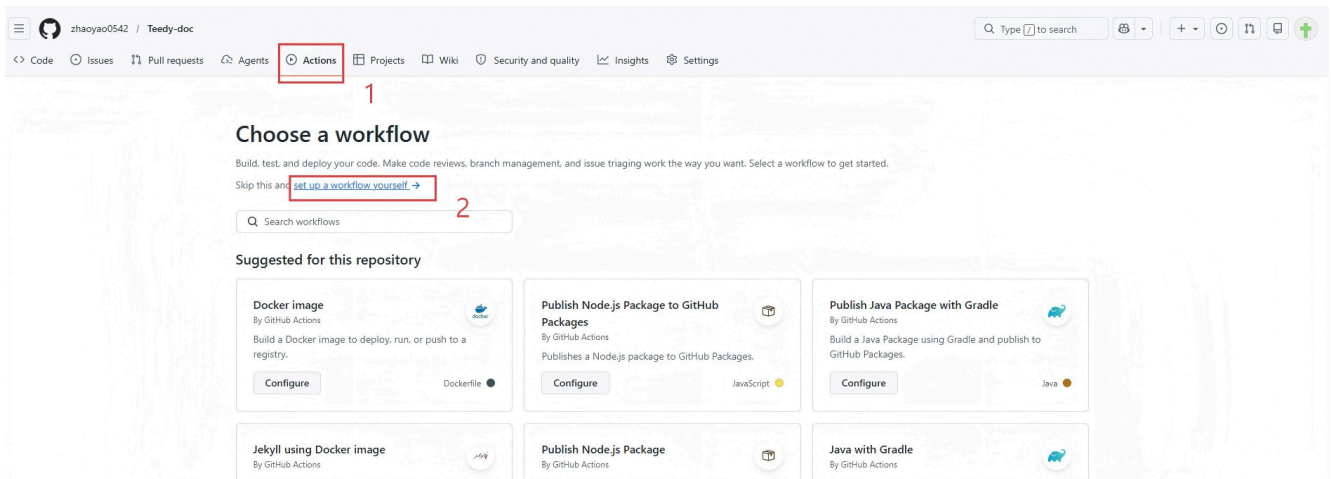
(打包下载全部文件)

Note: If archived HTML pages (like the Maven site) do not render correctly (missing CSS/JS), this is due to Jenkins' security restrictions. See [this workaround](#).

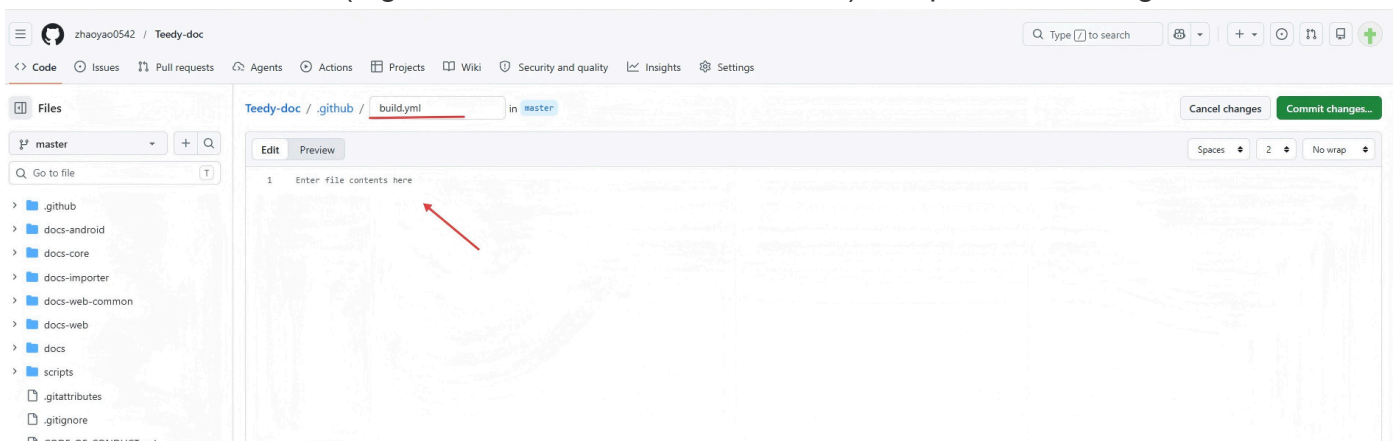
Part II: Github Actions

Create a workflow

1. In your GitHub repository, go to the **Actions** tab.
2. You can either:
 - choose a starter workflow (e.g., *Publish Java Package with Gradle*), or
 - click **set up a workflow yourself** to create a custom one.



3. Define the workflow file (e.g., `.github/workflows/build.yml`) and paste the configuration below.



Teedy's Simplified CI Workflow

Teedy's original workflow is at [build-deploy.yml](#). For this tutorial we use only the **build and publish** part (Docker is covered later).

```

name: Maven CI/CD

on:
  push:
    branches: [master]
    tags: [v*]
  workflow_dispatch:

jobs:
  build_and_publish:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 11
        uses: actions/setup-java@v4
        with:
          java-version: "11"
          distribution: "temurin"
          cache: maven
      - name: Install test dependencies
        run: sudo apt-get update && sudo apt-get -y -q --no-install-recommends install ffmpeg m...
      - name: Build with Maven
        run: mvn --batch-mode -Pprod clean install
      - name: Upload war artifact
        uses: actions/upload-artifact@v4
        with:
          name: docs-web-ci.war
          path: docs-web/target/docs*.war

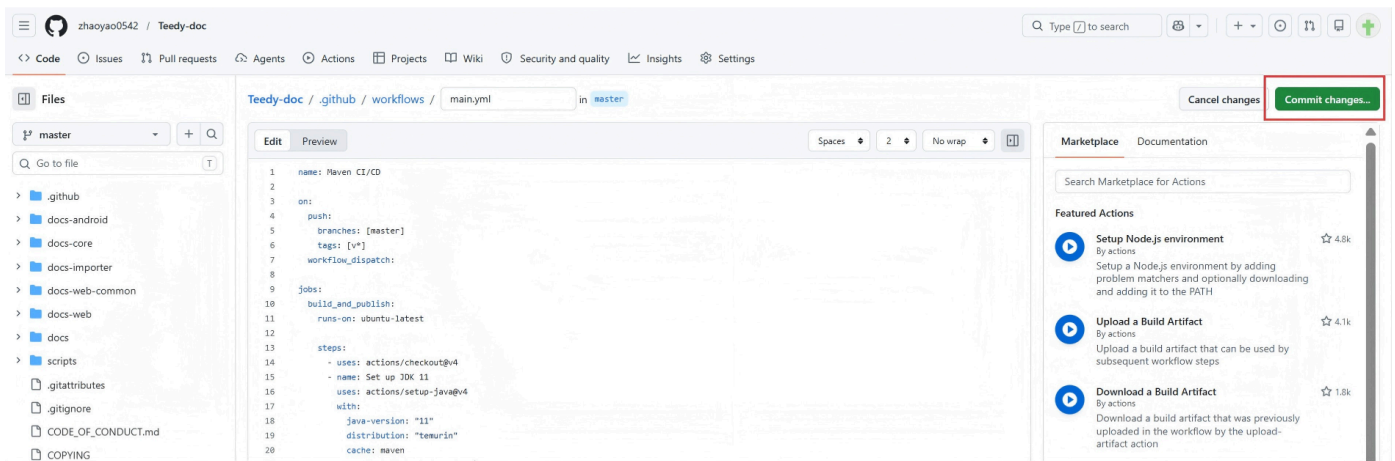
```

Key steps explained:

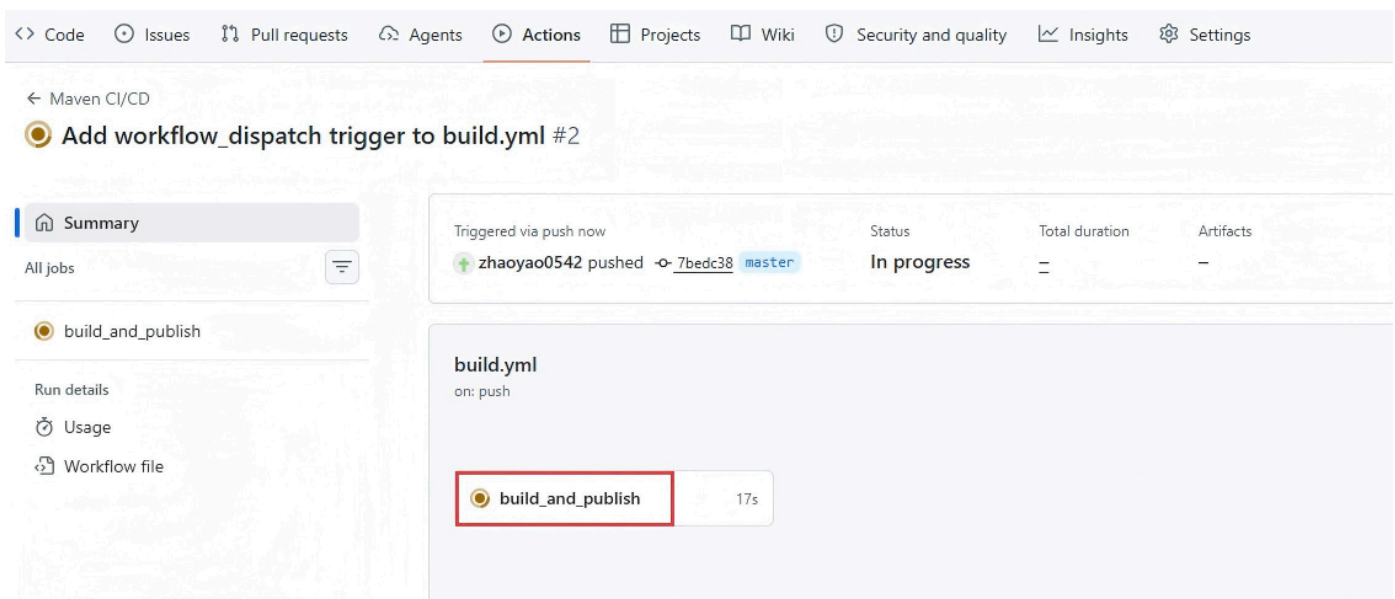
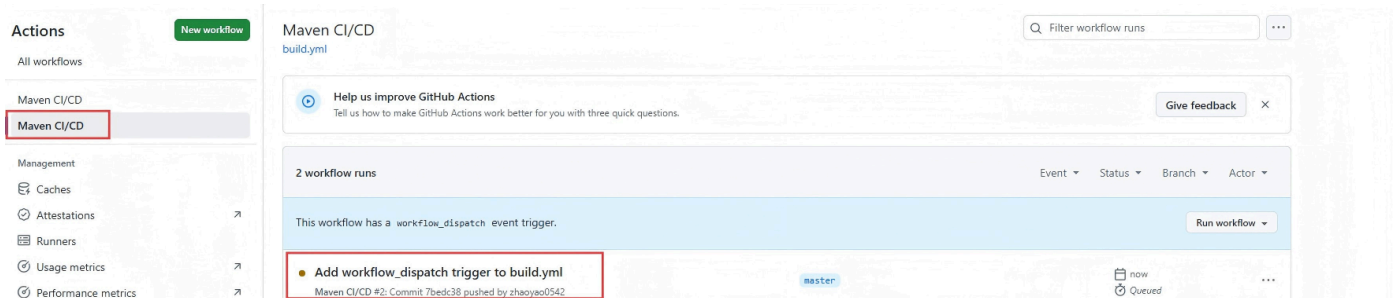
- **Trigger:** on push to `master` or new version tag, and manual dispatch.
- **Environment:** uses `ubuntu-latest` with JDK 11 (Temurin distribution).
- **Build:** runs Maven with the `prod` profile (production mode).
- **Artifact:** uploads the generated WAR file so you can download it from the Actions run page.

Commit and View Results

1. Paste above contents to the `.yaml` file, and click **Commit changes...**



2. Go to the **Actions** tab, you will see the workflow running.



Summary

All jobs

build_and_publish

Run details

Usage

Workflow file

build_and_publish

Started 40s ago

Build with Maven

```

50 [INFO]
51 [INFO] --- resources:3.4.0:resources (default-resources) @ docs-core ---
52 [INFO] Copying 45 resources from src/main/resources to target/classes
53 [INFO]
54 [INFO] --- compiler:3.15.0:compile (default-compile) @ docs-core ---
55 [INFO] Recompiling the module because of changed source code.
56 target 11] to target/classes
57 [INFO] /home/runner/work/Teedy-doc/Teedy-doc/docs-core/src/main/java/com/sismics/docs/core/uti
58 [INFO] /home/runner/work/Teedy-doc/Teedy-doc/docs-core/src/main/java/com/sismics/docs/core/uti
59 [INFO] /home/runner/work/Teedy-doc/Teedy-doc/docs-core/src/main/java/com/sismics/docs/core/dac
unchecked or unsafe operations.
60 [INFO] /home/runner/work/Teedy-doc/Teedy-doc/docs-core/src/main/java/com/sismics/docs/core/dac
61 [INFO]
62 [INFO] --- resources:3.4.0:testResources (default-testResources) @ docs-core ---
63 [INFO] Copying 19 resources from src/test/resources to target/test-classes
64 [INFO]
65 [INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ docs-core ---
66 [INFO] Recompiling the module because of changed dependency.
67 target 11] to target/test-classes
68 [INFO]
69 [INFO] --- surefire:3.0.0:test (default-test) @ docs-core ---
70 [INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
71 [INFO]
72 [INFO] -----
73 [INFO] T E S T S
74 [INFO] -----

```

☐ Upload war artifact
☐ Post Set up JDK 11
☐ Post Run actions/checkout@v4

3. After a successful run, you can download the `docs-web-ci.war` artifact from the workflow summary.

Summary

All jobs

build_and_publish

Run details

Usage

Workflow file

Triggered via push 2 hours ago

zhaoyao0542 pushed

fb1f8c8

master

Status

Success

Total duration

3m 50s

Artifacts

1

build.yml

on: push

build_and_publish

3m 46s

Annotations

1 warning

build_and_publish

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-java@v4, actions/upl

Show more

Artifacts

Produced during runtime

Name	Size	Digest
docs-web-ci.war	84.3 MB	sha256:29fb6046bcc5c2ec78a68afcfa00e92af6b69aa125fdc1ec4fe3172

Part III: Jenkins with Github Action

You can trigger a Jenkins pipeline directly from a GitHub Actions workflow. For a complete example, see the official tutorial:

[Build a Jenkins pipeline using Jenkinsfile Runner GitHub Actions](#)

This topic is optional but useful when you want to orchestrate Jenkins jobs from your GitHub repository.